

SFPPy Example 3: Advanced Migration Simulation with Variants

SFPPy Example 3: Advanced Migration Simulation with Variants

Overview

Variants Studied

Key Features of SFPPy in This Example

Step-by-Step Implementation

- 1 Define the Packaging Geometry 
- 2 Define the Migrant (Chemical Substance) 
- 3 Define the Polymer Layers 
- 4 Define Food Contact Conditions 
- 5 Chain the Food with the Packaging 
- 6 Simulate Mass Transfer Step-by-Step 

Variants & Comparisons

-  Variant 1: Replace Limonene with Toluene
-  Variant 2: Reduce PP Thickness
-  Variant 3: Combine Both
-  Compare All Conditions

Dependencies

Summary

Appendices

- Appendix . Available Geometries   in Module `geometry.py`
- Appendix . Implemented Layers Table  in module `layer.py`
- Appendix . Food Contact Properties Table  in module `foodlayer.py`

Overview

This example simulates the migration of **limonene** from a **recycled polypropylene (PP)** core to food, using a **trilayer packaging (ABA structure)** where **A = PET** and **B = PP**.

The study follows **three consecutive storage and processing steps**:

- 1 **Storage at ambient temperature (20°C) for 4 months**
- 2 **Hot-filling at 90°C with a fatty food**
- 3 **Storage at 30°C for 6 months**

The migration process is tracked for various conditions, including different **migrants (limonene/toluene)** and **layer thicknesses**.

Variants Studied 🧑🔬

#

- **Variant 1:** Replace limonene with toluene
- **Variant 2:** Reduce B's thickness from 20 µm to 10 µm
- **Variant 3:** Combine Variant 1 and Variant 2

The results are compared in a **single figure** and exported to **Excel/CSV** for further analysis.

Key Features of SFPPy in This Example 🚀

1. Database Interactions 📚

- **3D geometries** (predefined packaging shapes) 📦
- **Chemical substances** (via PubChem) 🧪
- **Food contact conditions** 🍽️
- **Polymers and materials** 🔬

2. Pythonic Syntax & Operators ⚡

- **>> Operator for Process Chaining:**
 - `mypackaging >> myfood >> mymaterial >> myfood`
 - Transfers **geometry, temperature, and mass transfer** in a single step.
- **+ Operator for Combining Results:**
 - Merge layers (`ABA = A + B + A`)
 - Combine simulation results (`fullsolution = foodcontact1.lastsimulation + foodcontact2.lastsimulation`)

3. Comprehensive Outputs 📊

- **Visual plots** 📈
 - **Data export to Excel/CSV** 💾
-

Step-by-Step Implementation 🔧

1 Define the Packaging Geometry 📦

#

```
from patankar.geometry import Packaging3D

container = Packaging3D('box_container', height=(8, "cm"), width=(10, "cm"), length=(19, "cm"))
```

- ♦ **3D shape:** Open rectangular prism
 - ♦ **Computed parameters:** Volume & Contact Surface Area
-

2 Define the Migrant (Chemical Substance)

#

```
from patankar.loadpubchem import migrant

m = migrant("limonene") # Retrieve properties from PubChem
```

- ♦ Chemical: Limonene
- ♦ Alternative (Variant 1): Toluene

3 Define the Polymer Layers

#

```
from patankar.layer import gPET, PP

A = gPET(l=(20, "um"), migrant=m, C0=0) # PET layer (no initial migrant)
B = PP(l=(0.5, "mm"), migrant=m, CP0=200) # PP layer with 200 mg/kg limonene

ABA = A + B + A # Create trilayer structure
```

- ♦ Structure: ABA (PET-PP-PET)
- ♦ Thicknesses: PET = 20 μm , PP = 500 μm

All layers are automatically populated with D and k values based on the chemical information available from **m** (migrant). Type **ABA** in the command line to see the details of the ABA structure.

```
[LAYER object version=1, contact=olivier.vitrac@agroparistech.fr]
3-multilayer of LAYER object:
-- [ layer 1 of 3 ] ----- barrier rank=2 -----
  name: "layer in gPET"
  type: "polymer"
  material: "glassy PET"
  code: "PET"
    l: 2e-05 [meter]
    D: 1.933e-17 [m**2/s]
      = Dpiringer(gPET,<migrant: DA-75016 - M=136.23 g/mol>,T=40.0 degC)
    k: 0.5494 [a.u.]
      = kFHP(<ethylene terephthalate>,<migrant: DA-75016 - M=136.23 g/mol>)
    C0: 0 [a.u.]
    T: 40 [degC]
-- [ layer 2 of 3 ] ----- barrier rank=3 -----
  name: "layer in PP"
  type: "polymer"
  material: "isotactic polypropylene"
  code: "PP"
    l: 0.0005 [meter]
    D: 4.258e-13 [m**2/s]
      = Dpiringer(PP,<migrant: DA-75016 - M=136.23 g/mol>,T=40.0 degC)
    k: 2.161 [a.u.]
      = kFHP(<propylene>,<migrant: DA-75016 - M=136.23 g/mol>)
    C0: 200 [a.u.]
    T: 40 [degC]
-- [ layer 3 of 3 ] ----- barrier rank=1 -----
```

```

name: "layer in gPET"
type: "polymer"
material: "glassy PET"
code: "PET"
  l: 2e-05 [meter]
  D: 1.933e-17 [m**2/s]
    = Dpiringer(gPET,<migrant: DA-75016 - M=136.23 g/mol>,T=40.0 degC)
  k: 0.5494 [a.u.]
    = kFHP(<ethylene terephthalate>,<migrant: DA-75016 - M=136.23 g/mol>)
C0: 0 [a.u.]
T: 40 [degC]

```

Our ABA technology

```
<multilayer with 3 layers: ['layer in gPET', 'layer in PP', 'layer in gPET']>
```

4 Define Food Contact Conditions

#

```

from patankar.food import realfood, liquid, fat, ambient, hotfilled, stacked

class contact1(stacked, ambient):
    name = "1:setoff"
    *`contacttemperature`* = (20, "degC")
    *`contacttime`* = (3, "months")

class contact2(hotfilled, realfood, liquid, fat):
    name = "2:hotfilling"

class contact3(ambient, realfood, liquid, fat):
    name = "3:storage"
    *`contacttime`* = (6, "months")

```

♦ Three contact phases:

- **contact1** : Storage (20°C, 4 months)
- **contact2** : Hot-filling (90°C)
- **contact3** : Storage (30°C, 6 months)

5 Chain the Food with the Packaging

#

```
container >> contact1 >> contact2 >> contact3
```

♦ Automatically propagates packaging **volume** and surface area to food contact conditions.

6 Simulate Mass Transfer Step-by-Step

#

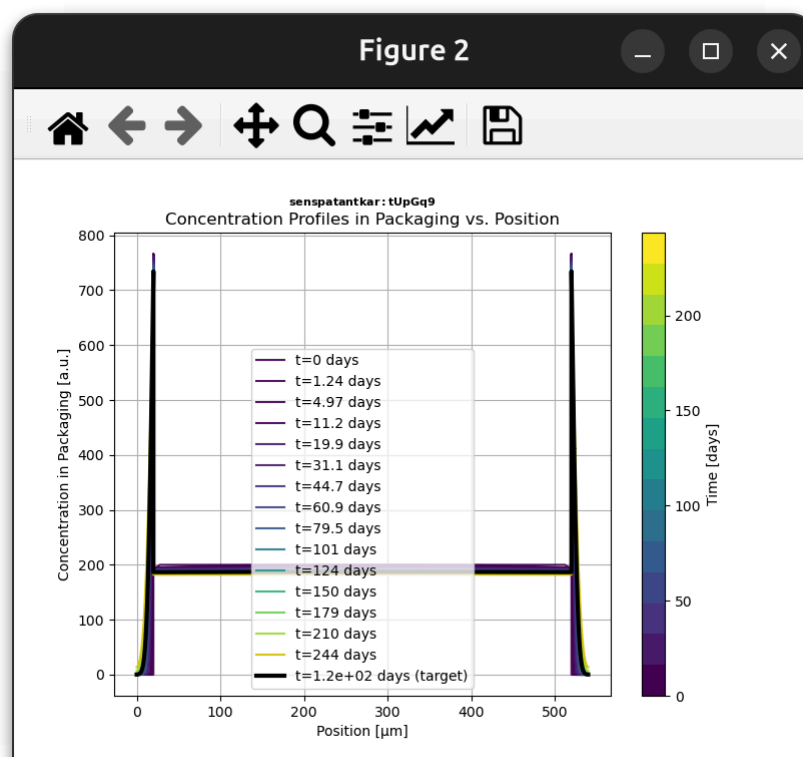
```
contact1 >> ABA >> contact1 >> contact2 >> contact3
```

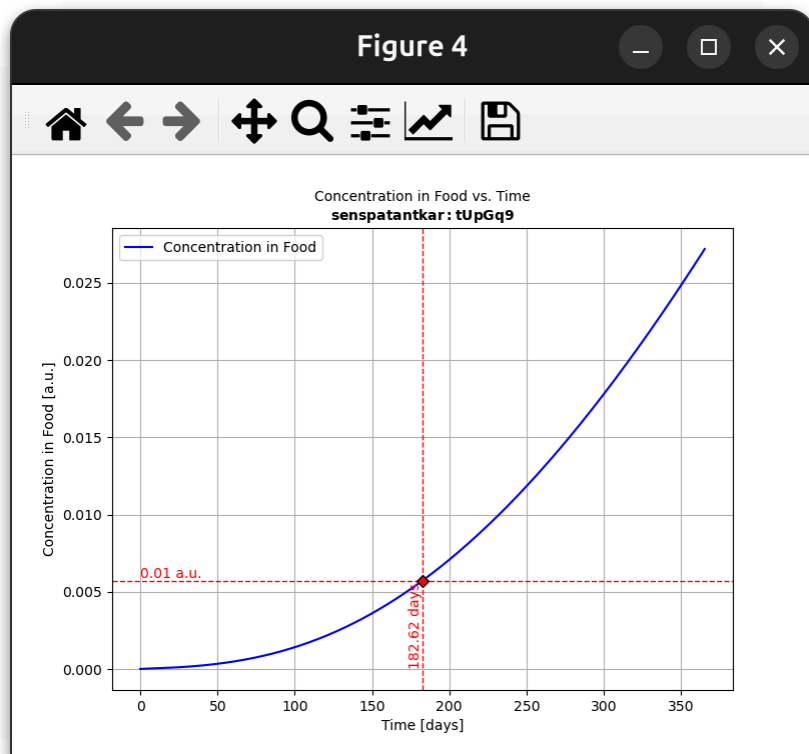
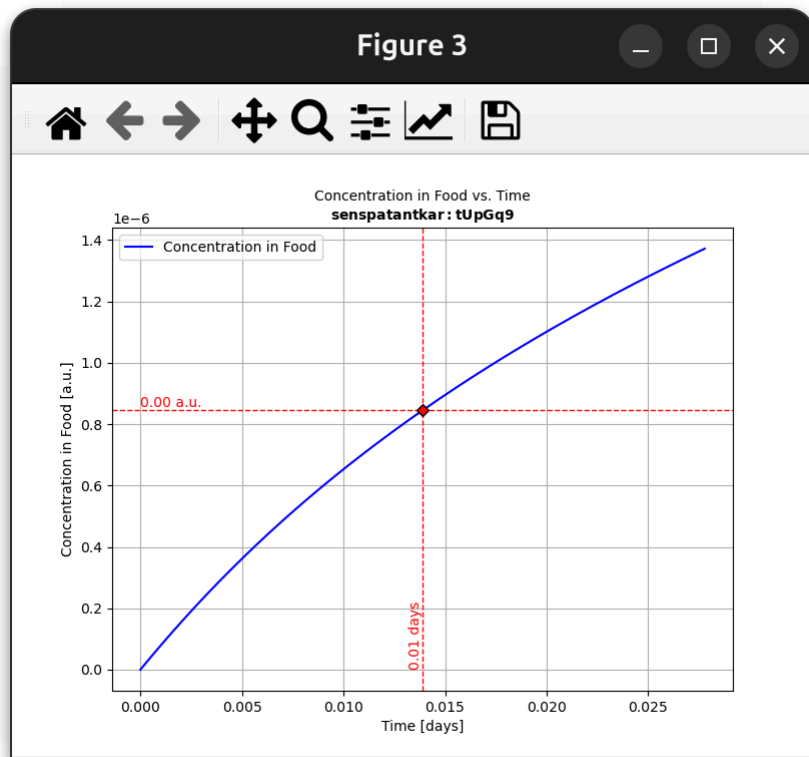
◆ Performs:

- Transfers temperature to the packaging
- Simulates mass transfer
- Propagates results to next steps

Plot migration kinetics:

```
contact1.lastsimulation.plotCx() # setoff condition  
contact2.lastsimulation.plotCF() # hot-filling  
contact3.lastsimulation.plotCF() # food storage
```





Variants & Comparisons

Variant 1: Replace Limonene with Toluene

#

```
m2 = migrant("toluene")
contact1 @ ABA.update(solute=m2) >> contact1 >> contact2 >> contact3
```

Variant 2: Reduce PP Thickness

#

```
newthickness = ABA.l.copy()
newthickness[[0, -1]] /= 2 # Reduce PET layers

contact1 >> ABA.copy(l=newthickness, migrant=m) >> contact1 >> contact2 >> contact3
```

Variant 3: Combine Both

#

```
contact1 @ ABA.copy(l=newthickness, migrant=m2) >> contact1 >> contact2 >> contact3
```

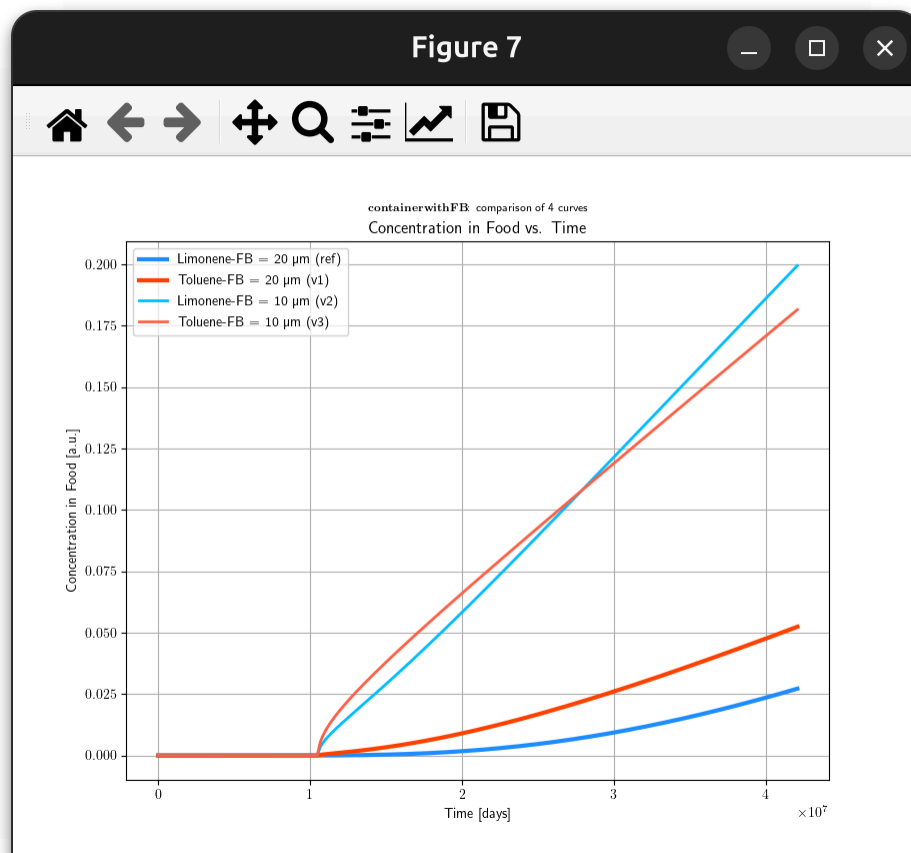
Compare All Conditions

#

```
from patankar.migration import CFSimulationContainer as store

collection = store(name="comparison")
collection.add(contact1.lastsimulation + contact2.lastsimulation + contact3.lastsimulation,
               "Reference")
collection.add(contact1.lastsimulation + contact2.lastsimulation + contact3.lastsimulation,
               "Variant 1")
collection.add(contact1.lastsimulation + contact2.lastsimulation + contact3.lastsimulation,
               "Variant 2")
collection.add(contact1.lastsimulation + contact2.lastsimulation + contact3.lastsimulation,
               "Variant 3")

collection.plotCF() # Plot all cases together
```



Dependencies

This example relies on several SFPPy modules:

Module	Purpose
<code>patankar.loadpubchem</code>	Retrieves chemical properties from PubChem 
<code>patankar.geometry</code>	Defines 3D packaging geometries 
<code>patankar.food</code>	Models food properties and storage conditions 
<code>patankar.layer</code>	Defines polymer layers and migration parameters 
<code>patankar.migration</code>	Solves mass transfer equations 

◆ Install required dependencies:

```
pip install numpy scipy pandas matplotlib
```


Summary

- ✓ Automated simulation chaining using `>>`
- ✓ Comparison of different migrants & thicknesses
- ✓ Dynamic updates using `update()`
- ✓ Plotting & exporting results

This example showcases **SFPPy's flexibility** in handling **multi-step migration simulations** with **minimal coding effort**. 

Next Steps:

- Run the code and explore **different materials, geometries, and food conditions!**
- Modify thicknesses and simulate new **barrier efficiency scenarios**.

Appendices

Appendix **1**. Available Geometries in Module `geometry.py`

#

 3D geometries are calculated by combining basic shapes covering:

Shape Type	Examples
Basic Shapes	  
Cylindrical Shapes	  
Open Containers	  
Prisms & Boxes	  
Spheres & Hemispheres	  
Pyramids	  

```
help_geometry()
```

Shape Class	Synonyms	Required Params	Description
Cone	cone	radius , height	A full cone with a closed circular base. Volume: $(1/3) \pi r^2 h$. Surface Area: Base + lateral area.
Cylinder	can , cylinder	radius , height	A standard cylinder with top and bottom faces. Volume: $\pi r^2 h$. Surface Area: Includes top and bottom disks.
Hemisphere	bowl , hemisphere	radius	A hemisphere (half a sphere) typically open at the flat side. Volume: $(2/3) \pi r^3$. Surface Area: $3\pi r^2$ ($2\pi r^2$ curved + πr^2 open).
OpenCone	open_cone	radius , height	A cone with its base removed, leaving a single open circular face. Volume: $(1/3) \pi r^2 h$. Surface Area: $\pi r \cdot \text{slant}$ (no base).
OpenCylinder1	glass , jar , open_cylinder_1 , pot	radius , height	A cylinder with exactly one open end (like a glass). Volume: $\pi r^2 h$. Surface Area: $2\pi rh + \pi r^2$.
OpenCylinder2	open_cylinder_2 , straw	radius , height	A cylinder with two open ends (like a straw). Volume: $\pi r^2 h$. Surface Area: $2\pi rh$ (no top or bottom).
OpenPrism1	box_container , open_prism1	length , width , height	A rectangular prism with ONE open face (e.g., open top). Volume: $l \cdot w \cdot h$. Surface Area: $2(lw + lh + wh) - lw$ (remove top).
OpenPrism2	open_prism2	length , width , height	A rectangular prism with TWO open faces (no top, no bottom). Volume: $l \cdot w \cdot h$. Surface Area: $2(lw + lh + wh) - 2(lw)$.
OpenSquare1	open_square1	side , height	A square-based box with ONE open face (like an open-top box). Volume: $\text{side}^2 \cdot \text{height}$. Surface Area: $\text{side}^2 + 4 \cdot \text{side} \cdot \text{height}$.

Shape Class	Synonyms	Required Params	Description
OpenSquare2	open_square2	side , height	A square-based box with TWO open faces (no top, no bottom). Volume: $\text{side}^2 * \text{height}$. Surface Area: $4 * \text{side} * \text{height}$.
RectangularPrism	box , cube , prism , rectangular_prism	length , width , height	A rectangular prism with all faces closed. Volume: $l * w * h$. Surface Area: $2(lw + lh + wh)$.
Sphere	sphere	radius	A full sphere. Volume: $(4/3) \pi r^3$. Surface Area: $4\pi r^2$.
SquarePyramid	square_pyramid	side , height	A square-based pyramid. Volume: $(\text{side}^2 * \text{height}) / 3$. Surface Area: Base + 4 triangles.

Appendix 2. Implemented Layers Table
in module
layer.py
#

The concept of **layer** refers to materials (usually dense materials), where mass transfer occurs by diffusion. Mass transfer in **layer** are described in **1D** by assuming that the packaging thickness is small comparatively to the food dimensions. Materials can associated in multiple layers and composite systems.

Layer Type	Included
Polymers & Materials	: thermoplastics (mono and copolymers)
Adhesives	: thermoplastics and thermosets
Multilayers	: laminates, coated with an arbitrary number of layers and thicknesses
Packaging Layers	: assembly potentially separated with an air layer

💡 **Note 1.** that layers can combined with operator **+** . The layer on the most left is assumed to be contact with food (food is on the left).

💣 **Note 2.** For risk assessment, we assume that properties D and k are set in a way the risk of mass transfer (*i.e.* , the value of the concentration in the food layer C_F) is maximized.

📄 **Note 3.** Food contact layers () are described in **0D** with the class **foodlayer** from the module **food.py** .

```
help_layer() # shows the table below
```

Class Name	Type	Material	Code
AdhesiveAcrylate	adhesive	acrylate adhesive	Acryl
AdhesiveEVA	adhesive	EVA adhesive	EVA
AdhesiveNaturalRubber	adhesive	natural rubber adhesive	rubber
AdhesivePU	adhesive	polyurethane adhesive	PU
AdhesivePVAC	adhesive	PVAc adhesive	PVAc
AdhesiveSyntheticRubber	adhesive	synthetic rubber adhesive	sRubber
AdhesiveVAE	adhesive	VAE adhesive	VAE
Cardboard	paper	cardboard	board
HDPE	polymer	high- <i>density</i> polyethylene	HDPE
HIPS	polymer	high-impact polystyrene	HIPS
LDPE	polymer	low- <i>density</i> polyethylene	LDPE
LLDPE	polymer	linear low- <i>density</i> polyethylene	LLDPE
PA6	polymer	polyamide 6	PA6
PA66	polymer	polyamide 6,6	PA6,6
PBT	polymer	polybutylene terephthalate	PBT
PEN	polymer	polyethylene naphthalate	PEN
PP	polymer	isotactic polypropylene	PP
PPrubber	polymer	atactic polypropylene	aPP
PS	polymer	polystyrene	PS
Paper	paper	paper	paper
SBS	polymer	styrene-based polymer SBS	SBS
air	air	ideal gas	gas
gPET	polymer	glassy PET	PET
oPP	polymer	bioriented polypropylene	oPP
plasticizedPVC	polymer	plasticized PVC	pPVC
rPET	polymer	rubbery PET	rPET
rigidPVC	polymer	rigid PVC	PVC

Graphical representation of classes dependence:



Appendix 3. Food Contact Properties Table in module `foodlayer.py` #

The  food,  contact conditions and  use are defined via keywords `foodphysics` , `fat` , `hotfilled` , , `sterilization`  covering;

Category	Suggested Emojis
General Physics	  
Chemical Affinity	  
Food Types	       
Storage Conditions	   
Simulants	   
Boundary Conditions	  

 **Note.** The `foodphysics` and `foodlayer` classes are the parent classes for all keywords. Several keywords can be combined together to generate new classes of food or food conditions. New classes can be derived directly by the user.

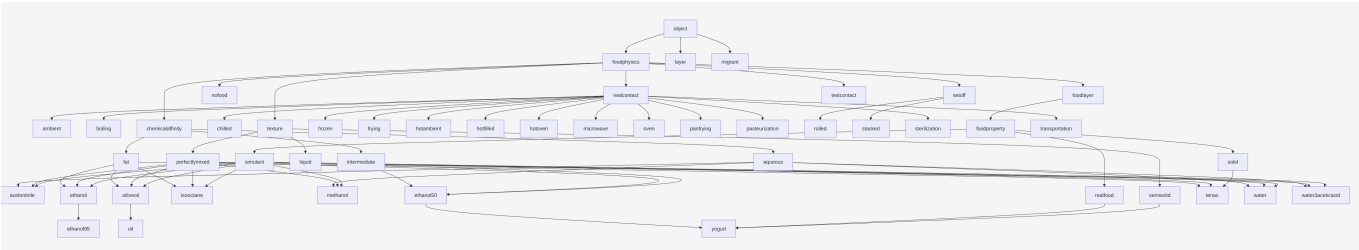
```
help_foodlayer()
```

Class Name	Name	Description	Level	Inheritance	Init Params
foodphysics	food physics	Root physics class used to implement food and mass transfer physics	base	object	
aqueous	aqueous contact	minimize mass transfer	root	chemicalaffinity	k
chemicalaffinity	undefined	default chemical affinity	root	foodphysics	k
fat	fat contact	maximize mass transfer	root	chemicalaffinity	k
foodlayer	generic food layer	root food class	root	foodphysics	$volume$, $surfacearea$, $density$, C_F0 , $contacttime$
intermediate	intermediate	intermediate chemical affinity	root	chemicalaffinity	k
liquid	liquid food	liquid food products	root	texture	h
nofood	undefined	impervious boundary condition	root	foodphysics	h
perfectlymixed	perfectly mixed	maximize mixing, minimize the mass transfer boundary layer	root	texture	h
realcontact	contact conditions	real storage conditions	root	foodphysics	$contacttemperature$, $contacttime$
semisolid	solid food	solid food products	root	texture	h
setoff	setoff	periodic boundary conditions	root	foodphysics	h
testcontact	migration testing	migration testing conditions	root	foodphysics	$contacttemperature$, $contacttime$
texture	undefined	default class texture	root	foodphysics	h
foodproperty	generic food layer	root food class	property	foodlayer	$volume$, $surfacearea$, $density$, C_F0 , $contacttime$
realfood	generic food layer	real food class	property	foodproperty	$volume$, $surfacearea$, $density$, C_F0 , $contacttime$
simulant	generic food simulant	food simulant	property	foodproperty	$volume$, $surfacearea$, $density$, C_F0 , $contacttime$
solid	solid food	solid food products	property	foodproperty	$volume$, $surfacearea$, $density$, h , C_F0 , $contacttime$

Class Name	Name	Description	Level	Inheritance	Init Params
ambient	ambient	ambient storage conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
boiling	boiling	boiling conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
chilled	ambient	ambient storage conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
frozen	frozen	freezing storage conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
frying	frying	frying conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
hotambient	hot ambient	hot ambient storage conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
hotfilled	hotfilled	hot-filling conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
hotoven	hot oven	hot oven conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
microwave	microwave	microwave-oven conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
oven	oven	oven conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
panfrying	panfrying	panfrying conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
pasteurization	pasteurization	pasteurization conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
sterilization	sterilization	sterilization conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
transportation	hot transportation	hot transportation storage conditions	contact	realcontact	<i>contacttemperature</i> , <i>contacttime</i>
acetonitrile	acetonitrile	acetonitrile	user	simulant, perfectlymixed, aqueous	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
ethanol	ethanol	ethanol = from pure ethanol down to ethanol 95%	user	simulant, perfectlymixed, fat	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
ethanol50	ethanol 50	ethanol 50, food simulant of dairy products	user	simulant, perfectlymixed, intermediate	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
ethanol95	ethanol	ethanol = from pure ethanol down to ethanol 95%	user	ethanol	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
isooctane	isooctane	isooctane food simulant	user	simulant, perfectlymixed, fat	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>

Class Name	Name	Description	Level	Inheritance	Init Params
methanol	methanol	methanol	user	simulant, perfectlymixed, aqueous	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
oil	olive oil	olive oil food simulant	user	oiliveoil	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
oiliveoil	olive oil	olive oil food simulant	user	simulant, perfectlymixed, fat	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
rolled	rolled	storage in rolls	user	setoff	<i>h</i>
stacked	stacked	storage in stacks	user	setoff	<i>h</i>
tenax	Tenax	simulant of dry food products	user	simulant, solid, fat	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
water	water	water food simulant	user	simulant, perfectlymixed, aqueous	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
water3aceticacid	water 3% acetic acid	water 3% acetic acid - simulant for acidic aqueous foods	user	simulant, perfectlymixed, aqueous	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>
yogurt	solid food	yogurt	user	realfood, semisolid, ethanol50	<i>volume</i> , <i>surfacearea</i> , <i>density</i> , <i>h</i> , <i>k</i> , <i>k</i> ₀ , <i>C_F0</i> , <i>contacttime</i>

Graphical representation of the classes dependence:



🌟 Happy Simulating! 🧪🔬



SFPPy for Food Contact Compliance and Risk Assessment

Contact [Olivier Vitrac](#) for questions | [Website](#) | [Documentation](#)